

Rev80

A Python desktop application that puts the core vibration analysis toolkit into the hands of plant managers and maintenance technicians, at a far more approachable price than hiring a dedicated engineering firm. It captures, analyzes, and records vibration data from industrial rotating equipment using IEPE accelerometers connected via a PicoScope 4000A USB oscilloscope.

Two ways to use it: on-demand snapshot capture (overall + spectral levels) for route-based motor inspection and installation verification, and **Monitor Mode** for continuous logging over hours or years, with anomaly detection to catch rare events like motor coast-down or vibration spikes.

Rev80 by Rev Engineering, LLC — 80% of the benefit of academic vibration analysis from a dedicated engineering firm, for a fraction of the cost.

Looking to build, package, or contribute to Rev80? See [CONTRIBUTING.md](#).

Table of Contents

- [Features](#)
 - [CLI Reference](#)
 - [Installing on Windows](#)
 - [Installing from Source \(any OS\)](#)
 - [Configuration and Persistence](#)
 - [Monitor Mode](#)
 - [Envelope-Demodulation Analysis \(Bearing Diagnostics\)](#)
 - [Signal Generator](#)
 - [Simulated Sensor](#)
 - [Architecture](#)
 - [Module Reference](#)
 - [Data Pipeline](#)
 - [AcquisitionSettings](#)
 - [VibeSample and ChannelResult](#)
 - [ScopeSensor](#)
 - [Data Storage](#)
 - [PicoScope Integration](#)
-

Features

- Real-time time-domain and frequency-domain plots across multiple simultaneous channels
- Welch-based power spectral density with configurable window (Hann, Blackman-Harris, Flattop, Hamming, etc.)
- Velocity and displacement spectra derived from acceleration via frequency-domain integration
- Single-shot and continuous streaming capture modes
- Per-channel 4th-order Butterworth highpass filter, causal with state carried across streaming blocks;
 `highpass_fc` is the declared band edge (response within $\pm 10\%$ there), not the -3 dB knee

- Mandatory hardware-level anti-aliasing (oversample + linear-phase Kaiser FIR decimate, measured -111.7 dB stopband) on every capture, plus a streaming-rate watchdog that flags degraded USB throughput
- Configurable IEPE sensor library: sensitivity (mV/EU), modality, engineering units
- PicoScope 4000A built-in signal generator for excitation testing
- HDF5 file save/load for post-processing and archiving (v4 format)
- Configurable frame cache depth (default 32 frames) with backward browse
- **Declared measurement band** — the overall is measured over a configurable band (default: highpass edge to F_{max}) with ISO 20816 presets, and the band is stored with the data so two readings can be compared
- **Linear power spectral averaging** over N frames, to pull small lines out of a noisy floor (off by default)
- **Envelope (demodulation) analysis** — band-pass around a structural resonance, Hilbert magnitude, envelope spectrum; the standard early-warning diagnostic for rolling-element bearing defects, with automatic demodulation-band selection
- **Crest factor and kurtosis** per channel — impulsiveness scalars that a broadband overall averages away
- Significance-based spectral peak selection: a line is reported when it stands a configurable number of dB above its own local noise floor, rather than by a fixed top- N amplitude ranking
- Trend plot: overall vibration amplitude over time per channel
- Simulated sensor for offline development and CI testing — a physically realistic bearing-defect model (impulse train at the defect rate ringing a structural resonance, load-zone amplitude modulation, slip jitter) with a healthy negative control
- Configurable amplitude modes: RMS, 0-P, P-P
- Configurable units: acceleration (g, mm/s², in/s²), velocity (mm/s, in/s, mil/s), displacement (mm, in, mil)
- **Monitor Mode** — interval datalogger: captures frames at a configurable interval (5 s – 2 days) into a single session HDF5; anomaly-triggered burst capture with two independent detection modes (EWMA-RMS broadband and fixed-level upper/lower thresholds; the EWMA-Spectral detector is temporarily unavailable in the GUI pending rework — see R39 — and remains reachable from the headless front end); configurable post-burst cooldown gate; manual “Record Burst” button
- **Session browser** — load and browse historical monitor sessions; burst events displayed as vertical markers on the vibration trend

CLI Reference

`rev80` is the single entry point for everything — GUI launch, the headless datalogger (as a subcommand), and quick info commands that need neither the GUI nor hardware. `rev80-headless` remains as a standalone shortcut for `rev80 headless` (handy for systemd units / scripts that only need the datalogger). Run `rev80 --help` or `rev80 headless --help` for the full, always-current option list.

```
rev80 --version           # print version and exit
rev80 --install-desktop-entry # Linux only – see Desktop integration below
rev80 --uninstall-desktop-entry
```

GUI mode

```
rev80                               # launch GUI
rev80 --from-file path/to/file.h5   # load measurement on startup
rev80 --from-file path/to/session_dir/ # load monitor session on startup
rev80 --debug                       # verbose logging
```

`--from-file` accepts a v4 single-measurement `.h5` file or a v5 monitor session directory (containing `session.h5`). The GUI opens, displays the data immediately, and the session browser and file browser remain fully functional.

Info commands

Return immediately — no GUI, no hardware, no heavy imports. Available both at the top level of `rev80` and under `rev80 headless` / `rev80-headless`:

```
rev80 --init-config      # seed config files and exit
rev80 --list-devices     # enumerate connected PicoScopes and exit
rev80 --list-sensors     # show the IEPE sensor library and exit
rev80 --edit-config      # open acquisition.yaml in $EDITOR and exit
```

Command	Description
<code>--init-config</code>	Create default config files and list them
<code>--list-devices</code>	Enumerate connected PicoScope devices
<code>--list-sensors</code>	Show the IEPE sensor library
<code>--edit-config</code>	Open <code>acquisition.yaml</code> in <code>\$EDITOR</code>

Headless mode

```
rev80 headless [options]
# or equivalently:
rev80-headless [options]
```

Before first use on a new machine, seed the config directory:

```
rev80 headless --init-config
```

This writes `acquisition.yaml` and `devices/picoscope-defaults.yaml` to `~/.config/rev80/` so you can edit them before connecting hardware.

The [info commands](#) above also work under `rev80 headless` / `rev80-headless`, in addition to the following headless-only options.

Session options:

Option	Default	Description
<code>--interval SECS</code>	600	Capture interval in seconds
<code>--pre-buffer SECS</code>	30	Pre-burst buffer duration
<code>--burst-duration SECS</code>	120	Burst capture duration
<code>--output DIR</code>	<code>~/Documents/Rev80/data/monitor/</code>	Output root directory
<code>--no-compress</code>	—	Disable gzip compression
<code>--start-now</code>	—	Skip the pre-start confirmation prompt

Acquisition options:

Option	Default	Description
<code>--device SERIAL</code>	auto-detect	PicoScope serial number, or <code>sim</code>
<code>--channels N [N ...]</code>	from device config	Channel indices to enable (persisted to device config)
<code>--maxfreq HZ</code>	from <code>acquisition.yaml</code>	Max analysis frequency override
<code>--binsize HZ</code>	from <code>acquisition.yaml</code>	Frequency resolution override
<code>--debug</code>	—	Verbose logging to stderr

First-run device config: On the first run with a new PicoScope, headless generates `devices/picoscope-<model>-<SN>.yaml` from the defaults template before the confirmation prompt. Edit it to set channel coupling, voltage range, and sensor assignments, then restart.

Channel persistence: `--channels 0 1` updates the `enabled` flag in the saved device config so the setting is sticky across restarts — you do not need to repeat the flag.

Confirmation gate: Before connecting, headless prints a session summary (device, channels, sample rate, monitor interval, anomaly config, config file paths) and waits for Enter. Use `--start-now` to bypass this for unattended use (systemd, cron, scripts).

Example workflows:

```
# First time on a new Pi – seed config, connect scope, generate device config
rev80-headless --init-config
rev80-headless --list-devices
rev80-headless                                # generates device config, shows summary

# Edit settings, then start unattended
nano ~/.config/rev80/acquisition.yaml
nano ~/.config/rev80/devices/picoscope-4424A-JY123.yaml
rev80-headless --start-now

# One-liner with explicit overrides (changes persisted to device config)
rev80-headless --channels 0 1 --interval 300 --start-now

# Simulated sensor – offline testing, no hardware
rev80-headless --device sim --interval 10 --start-now
```

Writes a `v5 session.h5` file loadable by the GUI session browser or `--from-file`. Clean shutdown on `Ctrl+C` or `SIGTERM` (suitable for systemd `Restart=on-failure`).

Installing on Windows

1. Install **PicoSDK 11.1.418** (or PicoScope 7 for Windows) from picotech.com/downloads. **Restart your computer** after installation so Windows registers the USB kernel driver.
2. Run `Rev80Setup-<version>.exe` and follow the installer. It creates a Start Menu shortcut and an uninstaller. No admin rights required.

SmartScreen warning: the installer is currently unsigned. Click *More info* → *Run anyway* to proceed.

Runtime file locations

Purpose	Location
Measurement data (.h5)	~/Documents/Rev80/data/
Log files	~/Documents/Rev80/logs/
Config / sensor library	%APPDATA%\rev80\

Installing from Source (any OS)

Works on Windows, Linux, and macOS. Requires Python 3.10+.

1. Install **PicoSDK** or **PicoScope 7** for your OS from picotech.com/downloads. **Restart your computer** after installation.
2. Get the source and install dependencies:

```
git clone <repo-url>/rev80.git # currently closed source. Contact RevEngineering.
cd rev80

# Runtime dependencies only
pip install .
```

3. Run the app:

```
# GUI (default)
python -m rev80

# GUI - open directly on a saved measurement or monitor session
python -m rev80 --from-file ~/Documents/Rev80/data/my_run.h5
python -m rev80 --from-file ~/Documents/Rev80/data/monitor/2026-06-02-130000/

# Headless interval datalogger (no display required)
# `rev80 headless ...` and `rev80-headless ...` are equivalent
rev80-headless --init-config          # seed config files first
rev80-headless                        # auto-detect scope, show summary
rev80-headless --device sim --interval 10 --start-now # offline test

# With debug logging to console
python -m rev80 --debug
```

Runtime dependencies: `numpy`, `scipy`, `dearpygui==2.0.0`, `h5py`, `pyyaml`, `plyer`, `picosdk`

Headless (no display) usage:

```
# Seed config on a fresh install
rev80-headless --init-config

# Auto-detect PicoScope - shows summary, waits for Enter
rev80-headless

# Fully explicit, skip prompt (suitable for scripts/systemd)
rev80-headless \
  --interval 300 \
```

```

--pre-buffer 30 \
--burst-duration 120 \
--output /mnt/nas/vibration \
--channels 0 1 \
--maxfreq 1000 \
--binsize 1 \
--start-now

# Simulated sensor (no hardware)
rev80-headless --device sim --interval 10 --start-now

```

Sessions written by the headless mode are identical v5 HDF5 files and can be loaded in the GUI:

```

# Open GUI on a specific session directory
python -m rev80 --from-file /mnt/nas/vibration/2026-06-02-130000/

# Or point at the session.h5 directly
python -m rev80 --from-file /mnt/nas/vibration/2026-06-02-130000/session.h5

# Or a regular single-measurement save
python -m rev80 --from-file ~/Documents/Rev80/data/my_measurement.h5

```

The `picosdk` package requires the PicoScope 4000A driver (`ps4000a.dll` / `libps4000a.so`) to be present on the system for hardware use. The app will start without it and show a “driver not found” notice in the device dialog — the simulated sensor is still available.

Desktop integration (Linux)

For a normal desktop install (not a dev checkout), use a **user-scheme pip install** — this places the `rev80` / `rev80-headless` console scripts in `~/.local/bin`, no venv or root required:

```

# Install the PicoScope driver (one-time, needs sudo – see drivers/)
sudo ./drivers/install-picoscope4000a-driver.sh

# Install rev80 itself into ~/.local/bin
pip install --user ".[gui]"

# Add a Rev80 entry to your application menu + an icon
rev80 --install-desktop-entry

# Remove it again
rev80 --uninstall-desktop-entry

```

`--install-desktop-entry` writes `~/.local/share/applications/rev80.desktop` and a set of icon PNGs under `~/.local/share/icons/hicolor/*/apps/rev80.png` (a fixed-size raster set, not the scalable SVG — Qt/KDE's SVG renderer doesn't render the source icon correctly, so this sidesteps it entirely), pointing `Exec=` at the exact `rev80` script that ran the install (works the same way from a venv). If `~/.local/bin` isn't already on your `PATH`, the command prints the line to add to `~/.bashrc` / `~/.profile` — most desktop distros add it by default, so this is usually a no-op. This is Linux-only; Windows gets a Start Menu shortcut from the [Inno Setup installer](#) instead.

Configuration and Persistence

Config lives in the OS-specific config directory: - **Linux/macOS:** `$XDG_CONFIG_HOME/rev80/` (default: `~/.config/rev80/`) - **Windows:** `%APPDATA%\rev80\`

Run `rev80 --init-config` (or `rev80-headless --init-config`) to create the directory and seed all default files. The layout is:

```
~/.config/rev80/
  acquisition.yaml           # acquisition + monitor settings (instance-wide)
  scope_sensors.yaml        # IEPE sensor library (shared across all devices)
  devices/
    picoscope-defaults.yaml  # channel template applied to new devices
    picoscope-4424A-JY123.yaml # per-device channel + siggen config
```

acquisition.yaml — instance-wide settings

Acquisition and monitor settings are shared across all scopes on this machine. Edit this file to change capture intervals, anomaly thresholds, filter settings, etc.

```
acquisition:
  maxfreq: 1000.0           # Hz — drives display sample rate (samplerate = 2.56 ×
                           maxfreq)
  binsize: 1.0              # Hz — drives FFT block size
  fft_window: hann
  welch_overlap: 0.5
  highpass_enabled: true
  highpass_fc: 10.0         # Hz
  trend_max_points: 5000
  cache_frames: 15

monitor:
  interval_s: 600           # seconds between interval captures
  pre_burst_s: 30           # seconds of pre-trigger data saved with each burst
  burst_duration_s: 120     # seconds of post-trigger burst capture
  max_burst_s: 600          # maximum burst length even if anomaly keeps retriggering
  output_dir: null          # null → ~/Documents/Rev80/data/monitor/
  compression: gzip
  compression_level: 4
  anomaly:
    enabled: true
    hook_type: rms           # rms | spectral | both
    warmup: 10              # frames before EWMA detection activates
    rms_pct: 10.0           # % deviation from EWMA baseline to trigger
    rms_s: 3.0              # seconds signal must stay above threshold before burst fires
    rms_alpha: 0.97         # EWMA smoothing (higher → slower baseline adaptation)
    spec_pct: 50.0          # % mean per-bin deviation from EWMA baseline to trigger
    spec_n: 10              # consecutive frames required for spectral trigger
    spec_alpha: 0.995       # very slow adaptation — spectral baseline changes slowly
    spec_fmin: null         # null = full spectrum; set Hz to restrict band
    spec_fmax: null
    fixed_upper_enabled: false # burst when overall amplitude rises above this level
    fixed_upper_value: 1.0
    fixed_upper_unit: in/s    # any unit in UNIT_TO_SI; converted automatically
    fixed_lower_enabled: false # burst when overall amplitude drops below this level
    fixed_lower_value: 0.05
    fixed_lower_unit: in/s
```

```
cooldown_enabled: false      # block re-triggers for this long after a burst fires
cooldown_s: 300.0
```

devices/picoscope-defaults.yaml — channel template

Applied to every channel when a new device is seen for the first time. Edit this before connecting a new scope to set your preferred defaults site-wide:

```
channel:
  enabled: false              # only channel 0 is enabled on new devices
  sensor_id: null
  voltage_range: 6            # PS4000A range index (6 = ±1 V)
  coupling: AC
  channel_name: null          # null → defaults to 'Ch A', 'Ch B', ...
  target_unit: null           # null → use sensor engineering units
  amplitude_mode: 0-P

siggen:
  enabled: false
  wave_type: PS4000A_SINE
  freq_hz: 1000.0
  pktpk_uv: 1000000           # 1 V pk-pk
  offset_uv: 0
```

devices/picoscope-<model>-<SN>.yaml — per-device channel config

Channel coupling, voltage range, sensor assignments, and signal generator settings for a specific scope. Generated automatically on first connection; edit to customise each channel.

```
channels:
  0:
    enabled: true
    sensor_id: <uuid>          # from scope_sensors.yaml
    voltage_range: 6           # ±1 V
    coupling: AC
    channel_name: Motor NDE
    target_unit: in/s
    amplitude_mode: 0-P
  1:
    enabled: false
    sensor_id: null
    voltage_range: 6
    coupling: AC
    channel_name: null
    target_unit: null
    amplitude_mode: 0-P
siggen:
  enabled: false
  wave_type: PS4000A_SINE
  freq_hz: 1000.0
  pktpk_uv: 1000000
  offset_uv: 0
```

scope_sensors.yaml — global IEPE sensor library

User-defined IEPE sensors shared across all devices. Add entries here to make sensors available for assignment in the GUI Channel Config panel or headless config:

```
- id: <uuid>
  name: PCB 352C33 Ch1
  sensitivity: 10.2 # mV per engineering unit
  engineering_units: g
  notes: ''
```

Managed via `ScopeSensorRegistry` — provides CRUD operations. When a sensor is assigned to a channel, `DataCollector` divides incoming mV by `sensitivity` to produce engineering units.

Logging

Logging is configured via `src/rev80/logging.yaml`. In development, log files are written to `log/`. In a frozen Windows build, logs go to `~/Documents/Rev80/logs/`.

Monitor Mode

Monitor Mode turns Rev80 into a continuous interval datalogger with automatic event capture.

Interval recording

When monitoring is active, the controller records a frame at the configured interval into `/monitor/{N}/` in `session.h5`. Interval captures continue regardless of whether a burst is in progress.

Each stored frame is a full-bandwidth raw-rate capture (see [Data Storage](#)), so total storage scales with interval, not with `maxfreq`. The Monitor config dialog shows an estimated bytes/year figure as you adjust the interval; it turns red with a ⚠ warning icon above 10 GB/year — a signal to reconsider the interval before arming a long run, not a hard limit.

Burst capture

Any trigger source — automatic anomaly detection or the manual **Record Burst** button — causes the controller to:

1. Snapshot `pre_buffer_s` of raw frames from the ring cache and prepend them to the burst as pre-trigger data.
2. Capture frames at full rate for `burst_duration_s` seconds, writing to `/burst/{id}/`.
3. Tag the t=0 frame (anomaly onset, not detection time) in `burst.attrs` as `trigger_timestamp / trigger_rel_time`.
4. Start the cooldown gate to block further automatic triggers for `cooldown_s` seconds.

Anomaly detection hooks

Three hook types can be used independently or combined (configured by the “Hook: RMS / Spectral / Both” selector):

Broadband EWMA (RMS)

Maintains a per-channel EWMA baseline of the overall broadband amplitude. Triggers when:

```
|current - baseline| / baseline > rms_pct / 100
```

...for `rms_s` seconds of sustained deviation. The baseline adapts continuously during normal operation and during burst playback. A warmup period (`warmup` frames) must elapse before triggering is enabled.

Parameter	Default	Description
<code>rms_pct</code>	10.0	% deviation from EWMA baseline
<code>rms_s</code>	3.0	Seconds signal must stay above threshold (0 = first frame)
<code>rms_alpha</code>	0.97	EWMA smoothing factor (higher = slower baseline)
<code>warmup</code>	10	Frames to collect before triggers are enabled

Frequency-shape EWMA (Spectral)

Compares the live PSD against a per-bin EWMA baseline. Triggers when the mean spectral deviation across the monitored frequency band exceeds `spec_pct` for `spec_n` consecutive frames. Useful for detecting new harmonics or bearing-tone shifts that don't change overall amplitude significantly.

Parameter	Default	Description
<code>spec_pct</code>	50.0	% mean per-bin deviation to trigger
<code>spec_n</code>	10	Consecutive frames required
<code>spec_fmin</code> / <code>spec_fmax</code>	null	Restrict band (null = full spectrum)

Use **Reset Baseline** (monitor card button) to reseed the EWMA from the current frame after a process change, speed change, or restart.

Absolute level trigger (Fixed)

Fires immediately (no warmup, no EWMA) when the overall amplitude crosses a fixed level. Accepts any supported unit and converts automatically — a threshold in `in/s` works correctly against a channel reporting `mm/s`.

Upper and lower limits are independent. A channel with no sensor/EU assigned (raw `mV`) logs a one-time warning and is silently skipped.

Parameter	Description
<code>fixed_upper_enabled</code> / <code>fixed_upper_value</code> / <code>fixed_upper_unit</code>	Trigger when amplitude rises above this level
<code>fixed_lower_enabled</code> / <code>fixed_lower_value</code> / <code>fixed_lower_unit</code>	Trigger when amplitude falls below this level

Cooldown gate

After any burst fires (automatic or manual), the controller optionally blocks further automatic triggers for `cooldown_s` seconds. Interval captures are unaffected. Use this to prevent a sustained fault from generating many overlapping burst files.

Envelope-Demodulation Analysis (Bearing Diagnostics)

The **Envelope** tab is for one specific job: catching rolling-element bearing defects — inner race, outer race, ball, or cage — early, before they show up as broadband vibration. It's off by default (enable it in the Acquisition dialog under **Analysis Tabs** → **Envelope/Demodulation (bearing analysis)**).

Why the raw spectrum misses this

A bearing defect doesn't announce itself as a clean line at its own defect frequency. Every time a rolling element rolls over the defect it produces a sharp mechanical impulse, and that impulse rings a structural resonance of the bearing housing — typically somewhere in the 2–20 kHz range, well above anything the machine itself does mechanically (running speed, gear mesh, blade pass). In the raw spectrum that impulse energy is smeared thinly across the whole width of that resonance, and it sits *underneath* the 1x running-speed line and its harmonics, which are usually orders of magnitude larger. You can stare at the raw spectrum with a fresh defect right in front of you and see nothing but the same 1x/2x/3x peaks you always see.

What actually carries the diagnosis isn't *where* the energy is — it's *how it's modulated*. The impulses repeat at a steady rate (the defect rate), so the resonance's amplitude keeps swelling and dying back at that rate. Recovering that swell-and-die pattern — the **envelope** — turns “energy smeared across a resonance” into a clean, discrete line at the defect rate, usually with sidebands at $\pm 1x$ (running speed) on either side from the load zone modulating the impact severity as the shaft turns. This is the same technique sold as CSI PeakVue, SKF Enveloped acceleration (gE), or “shock pulse” analysis — the name changes, the physics doesn't.

Reading the envelope spectrum

The Envelope plot looks like an ordinary spectrum, but the x-axis is **modulation frequency**, not vibration frequency, and the content on it means something different:

- **A clean line at (or near) a known bearing defect frequency is the signature.** If you have the bearing's geometry (or a manufacturer table), compute BPFO, BPFI, BSF, or FTF for the actual running speed and compare — a line within a percent or two of one of those, that wasn't there on a healthy baseline, is the finding. Rev80 doesn't compute these for you; it gives you the clean line to compare against numbers you already have.
- **$\pm 1x$ sidebands around a defect line** (spaced at running speed) are corroborating, not optional extra credit — they're the mechanism (load-zone modulation) showing up exactly where theory says it should. A defect-rate line with no sidebands is worth a second look before you commit to a bearing call.
- **A rising forest of harmonics of the defect rate** (1x, 2x, 3x... of BPFO, say) as a fault progresses from a point defect toward spalling is a normal severity progression — more harmonics and a higher noise floor between them, not just a taller first line.
- **A line at 1x running speed itself** (not a defect frequency) usually means the demodulation band leaked some of the machine's own vibration through the band-pass — see below — rather than a real finding.
- **No lines at all, just a low, structureless floor** is the actual healthy-bearing picture. Resist the urge to read something into floor texture; the whole point of this technique is that a real defect stands out as a discrete line, not a shape.

Trending matters here as much as it does for overall amplitude: a defect-rate line that grows session over session is a much stronger case than a single reading, and it's how you tell a marginal-but-stable indication from one that's headed toward failure.

Choosing the demodulation band

The **Band** fields set the low/high edges (Hz) of the band-pass filter applied before demodulation — this is the single most consequential choice in the whole technique, because everything downstream is only as good as the resonance you picked:

- **Leave both fields at 0 and click Auto**, or leave them at 0 and just let it re-run every frame — this searches the upper 75% of your configured frequency range (above $F_{\text{max}}/4$) for the frequency region carrying the most energy, smoothed over the width of the proposed band so a single tall harmonic can't fool it into centering on machine content instead of a resonance. This is deliberately restricted to the *upper* part of the range: the whole reason to demodulate is to escape the 1x/2x/gear-mesh content that dominates the lower part, so a band centered down there would just recover that content again, which is worse than doing nothing.
- **Type an explicit band once you know where the resonance actually is.** The Auto suggestion is a reasonable first look, not necessarily the right answer — a housing or bearing has more than one structural resonance, and the one that rings loudest under a hammer tap or a bump test isn't guaranteed to be the one Auto finds from operating data alone. If you've identified the real resonance (bump test, or Auto's suggestion drifting frame to frame because there isn't one dominant peak), type it in and it stays fixed.
- **Width matters as much as center.** Too narrow and you lose modulation sidebands and impulse energy the resonance actually carries — the band needs to be wide enough to pass the resonance's own bandwidth, typically several hundred Hz to a few kHz depending on how lightly damped it is. Too wide and you start letting the 1x/2x machine lines back in at the band edges, which shows up as that spurious 1x line in the envelope mentioned above.
- **The band must sit strictly inside the acquisition range** ($0 < \text{low} < \text{high} < \text{Nyquist}$) and — since a bearing resonance is a structural, not running-speed-dependent, frequency — well above your highest expected running-speed harmonic. Acquisition always runs at a fixed rate independent of F_{max} — 25.6 kHz, Nyquist 12.8 kHz — specifically so the whole 2–20 kHz range a housing resonance can live in is always available to demodulate, regardless of what F_{max} you have the Spectrum tab set to. F_{max} only controls what the *Spectrum* tab displays; it has no effect on what Envelope can see.
- **Rev80 still warns you if this ever isn't the case:** the Envelope tab checks the actual Nyquist of the data it's demodulating and shows a banner if it's implausibly low for a resonance to fit under. In normal use this should never fire — it exists as a safety net (e.g. if a future change lowers the acquisition rate) rather than something you'll routinely hit by choosing a low F_{max} .
- The info line under the Band controls reports the resolved band and the resulting envelope's frequency ceiling for the current frame, so you can see what Auto actually picked.

Signal Generator

The PicoScope 4000A has a built-in arbitrary waveform generator (AWG) on its AUX output. Rev80 exposes this through the **Generate** config tab.

Setting	Description
Enabled	Enable/disable AWG output
Waveform	Sine, Square, Triangle, DC voltage, Ramp Up/Down
Frequency (Hz)	Output frequency
Amplitude (mV pk-pk)	Peak-to-peak voltage
Offset (mV)	DC offset

Timing: The signal generator runs **continuously** from stream start to stream stop — it is programmed once when the stream starts, free-running, with no per-block triggering. The AWG and the ADC acquisition run independently and simultaneously.

Signal generator settings are persisted per-device (see [Configuration and Persistence](#)) and restored automatically when the same device reconnects.

Simulated Sensor

`VibeSensor.simulated()` returns a `VibeSensor` with `is_simulation=True`. When connected, it starts a `SimulatedSensor` daemon thread that generates synthetic bearing-defect vibration data at the raw acquisition rate (`raw_samplerate / raw_blocksize`) — the same rate `PicoScopeStream` reports, so the simulated path exercises the same raw/display decimation everything else does rather than silently skipping it. Pass `--device sim` (headless) or select the simulated sensor in the GUI's device dialog — no PicoScope hardware required.

`SimulatedSensor` is **excluded from `VibeSensor.find()`** — it will never appear in the hardware device list. Use `VibeSensor.simulated()` directly in tests and offline development.

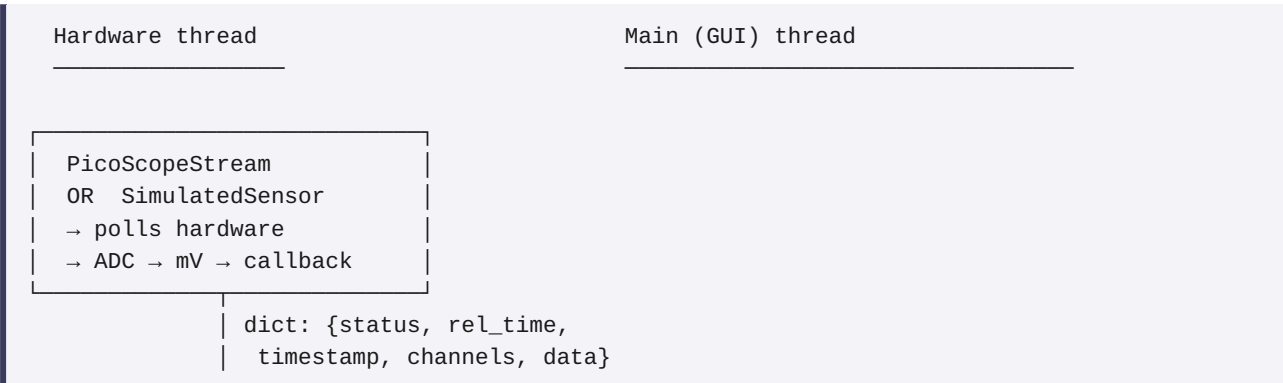
Available generators in `simulation.py`:

Generator	Description
<code>GenerateTone(config, ampl, freq, phase)</code>	Pure sinusoid
<code>GenerateNoise(config, ampl)</code>	White Gaussian noise
<code>GenerateBearingVibration_SpectralMethod(config)</code>	Bearing fault signal assembled in frequency domain (exponential noise floor + running harmonics + bearing-fault sidebands)
<code>GenerateBearingVibration_TemporalMethod(config)</code>	Bearing fault signal assembled in time domain (noise + harmonics with phase variation)

The default source for `SimulatedSensor` is `GenerateBearingVibration_TemporalMethod`. The source can be overridden at construction for targeted unit testing.

Architecture

The pipeline is strictly layered. The hardware thread and the GUI render loop are decoupled via a `threading.Event` — the collector never imports or calls into DPG. All DSP (Welch FFT, filtering, integration) lives in `collector.py`; `gui.py` is a pure presentation layer that consumes pre-computed `ChannelResult` objects.





Decoupling properties: - `collector.py`, `monitor/`, `sample.py`, `picoscope.py` — zero DPG imports. Any event loop can drive them. - `new_frame_event` is a `threading.Event` — a stdlib primitive with no GUI dependency. - `MonitorWriterThread` runs as a daemon thread independent of both the hardware thread and GUI. - A headless process can replace the GUI by polling `new_frame_event`, calling `process_samples()`, and passing results to `MonitorController` — approximately 50 lines.

When the GUI is slower than the hardware data rate it skips to the latest frame — earlier frames remain in the ring cache (default 32 frames) for browsing. The hardware thread is never blocked by rendering.

Module Reference

Module	Responsibility
<code>__main__.py</code>	rev80 CLI entry point — top-level info commands, <code>--version</code> , <code>headless</code> subcommand dispatch, GUI launch (logging setup, GUI instantiation, main loop, cleanup)

Module	Responsibility
<code>desktop.py</code>	Linux desktop integration — <code>install()</code> / <code>uninstall()</code> write/remove <code>~/.local/share/applications/rev80.desktop</code> + icon, driven by <code>rev80 --install-desktop-entry / --uninstall-desktop-entry</code>
<code>_paths.py</code>	Runtime-safe resource/data/log directory resolution — editable checkout, non-editable pip install, and frozen (PyInstaller) builds all resolve correctly; see Configuration and Persistence
<code>logger.py</code>	YAML-configured logging (<code>logging.yaml</code>); writes to <code>log/</code> ; global exception hook
<code>util.py</code>	Constants (<code>MAXFREQ_PRESETS</code> , <code>BINSIZE_PRESETS</code> , <code>UNITS</code> , <code>AMPLITUDE_MODES</code> , <code>MONITOR_INTERVAL_PRESETS</code>), unit taxonomy and SI conversion, integration order helpers, <code>UI_Elements</code> DPG tag registry
<code>icons.py</code>	CommitMono Nerd Font (Codicons) registry; <code>load()</code> registers font with DPG; <code>IC</code> dict maps icon names to <code>\uXXXX</code> codepoints
<code>sensor.py</code>	<code>VibeSensor</code> dataclass — device metadata; <code>find()</code> enumerates hardware PicoScopes only; <code>simulated()</code> returns a test sensor; <code>connect()</code> returns the appropriate stream
<code>picoscope.py</code>	<code>FindPicoScope()</code> — enumerates PS4000A units; <code>PicoScopeStream</code> — polling thread, ADC → mV, overflow detection, anti-alias oversample/decimate (<code>antialias_decimate()</code>), streaming-rate degradation watchdog, silence-watchdog recovery, signal generator setup
<code>scope_sensor.py</code>	<code>ScopeSensor</code> dataclass — IEPE sensor metadata: name, sensitivity (mV/EU), engineering units, amplitude mode, UUID
<code>scope_sensor_registry.py</code>	<code>ScopeSensorRegistry</code> — YAML-backed CRUD for user sensor library and per-channel assignments
<code>_dsp.py</code>	Windowing and band helpers for frequency-domain integration: Hann/Tukey tapers with their measured error tables, <code>band_mask()</code> , <code>integrate_rfft()</code> , <code>butter_knee_for_edge()</code> , <code>crest_factor()</code> , <code>kurtosis()</code>
<code>peaks.py</code>	Significance-based spectral peak selection — per-bin local noise floor via a running median, array-valued <code>height</code> / <code>prominence</code> into a single <code>find_peaks</code> call
<code>envelope.py</code>	Envelope (demodulation) analysis — <code>envelope_spectrum()</code> (band-pass → Hilbert magnitude → DC removal → amplitude spectrum) and <code>suggest_band()</code> for automatic demodulation-band selection

Module	Responsibility
<code>sample.py</code>	<code>AcquisitionSettings</code> — spectrum, filter, band, averaging and cache config with derived properties; <code>VibeSample</code> — single-channel time-domain block; <code>ChannelResult</code> — frozen display-ready result from <code>process_sample()</code>
<code>config.py</code>	OS-aware config directory; per-device YAML persistence (channels, acquisition settings, monitor defaults); atomic writes; fallback to built-in defaults
<code>collector.py</code>	<code>DataCollector</code> — multi-channel acquisition state machine: stream lifecycle, per-channel causal Butterworth highpass filtering with state carried across blocks, mV → EU conversion, frame ring cache, <code>new_frame_event</code> signal, DSP via <code>process_sample()</code> / <code>process_samples()</code> , trend accumulation, HDF5 save/load, monitor session loaders
<code>simulation.py</code>	<code>SimulatedSensor</code> (daemon thread, paced against a deadline) + signal generators. <code>GenerateBearingVibration</code> is the default: a physically realistic defect model (impulse train, resonance carrier, load-zone AM, slip jitter) with <code>severity=0</code> as the healthy control. <code>GenerateTone</code> , <code>GenerateNoise</code> and the two older pure-tone generators remain for regression coverage
<code>gui.py</code>	<code>GUI</code> class — dearpygui three-column layout with manual render loop (<code>_poll_new_frames</code>), all config dialogs, spectrum/time/trend plots, file I/O, monitor card, session browser
<code>monitor/__init__.py</code>	Re-exports: <code>MonitorController</code> , <code>MonitorSession</code>
<code>monitor/session.py</code>	<code>MonitorSession</code> frozen dataclass — session parameters, config snapshots, and cooldown settings
<code>monitor/gate.py</code>	<code>IntervalGate</code> — snap-to-grid capture scheduler; burst mode entry/exit; caller-supplied time (unit-testable)
<code>monitor/anomaly.py</code>	<code>AnomalyHook</code> Protocol; <code>NullAnomalyHook</code> ; <code>RmsThresholdHook</code> ; <code>SpectralThresholdHook</code> ; <code>FixedThresholdHook</code> ; <code>CompositeAnomalyHook</code> ; <code>AnomalyEvent</code> dataclass
<code>monitor/writer.py</code>	<code>MonitorWriterThread</code> — daemon thread; appends interval frames to <code>/monitor/</code> and burst frames to <code>/burst/</code> in <code>session.h5</code> ; disk-space guard
<code>monitor/controller.py</code>	<code>MonitorController</code> — owns gate, writer, anomaly hook; anomaly hook supplied at <code>start()</code> and active for the full session; <code>trigger_burst()</code> for manual burst

Data Pipeline

1. Acquisition — `PicoScopeStream` (`picoscope.py`)

Acquisition and display run at two independent rates. `PicoScopeStream` always acquires at a fixed rate (`AcquisitionSettings.raw_samplerate`, see `RAW_SAMPLERATE_HZ` in `sample.py` — 25.6 kHz, 12.8 kHz Nyquist by default) high enough to always contain a bearing housing resonance (typically 2–20 kHz), completely independent of the user's chosen `maxfreq`. `maxfreq` only controls what gets *displayed* — the Spectrum tab's rate and the HDF5 storage cost still track it (see `samplerate / blocksize` below), but the signal itself is always captured, stored, and available to envelope analysis at the full raw rate. This is the same approach other route-based vibration analyzers use: display capped low per ISO monitoring convention, demodulation still has full bandwidth underneath.

`PicoScopeStream` is a background polling thread that wraps `ps4000aRunStreaming`. The ADC is always driven faster than `raw_samplerate` — at an oversampling ratio (up to 4×, capped by a measured safe continuous-streaming ceiling for this hardware) — so a linear-phase Kaiser-windowed FIR anti-alias filter (designed for ≥100 dB stopband; measured −111.7 dB worst case, against −60.0 dB for scipy's default Hamming kernel) can reject content above `raw_samplerate`'s Nyquist *before* decimating down to it; this is mandatory and not user-configurable. On each poll:

1. Converts ADC counts → mV via `adc2mV()` for all enabled channels
2. Detects ADC overflow per channel via the overflow bitmask
3. Accumulates mV samples in a raw, oversampled (`raw_blocksize × effective_osr × N_channels`) buffer
4. Once a full raw block has accumulated, anti-alias filters and decimates it down to `raw_blocksize` samples via `antialias_decimate()`, then fires the registered callback with:

```
{
    'status':          str,          # 'OKAY', 'OVERFLOW', etc.
    'overflow_mask':   int,          # bitmask, one bit per channel
    'degraded':         bool,        # True if sustained USB streaming throughput has
    # fallen below the rate watchdog's threshold
    'rel_time':        float,        # seconds since stream start
    'timestamp':        datetime,
    'unit':             ['mV', ...], # one entry per channel
    'channels':         [0, 1, ...], # enabled channel indices
    'data':             ndarray,     # shape (raw_blocksize, N_channels), mV
    'samplerate':       float,        # raw_samplerate (or driver-reported actual rate)
}
```

A watchdog thread monitors for >5 s silence and attempts up to 3 reconnect cycles automatically. A second, independent watchdog monitors *sustained* USB streaming throughput every 2 s — this catches a different failure mode where the driver silently delivers only a fraction of the requested rate (`status='OKAY'`, no overflow, callbacks keep firing) that the silence watchdog can't see. It only sets `degraded=True` on the stream and logs a warning; it never triggers reconnect, since a bandwidth ceiling isn't fixed by reopening the device.

Device discovery — `VibeSensor.find()` calls `FindPicoScope()` and returns hardware-only results.

`SimulatedSensor` is excluded; use `VibeSensor.simulated()` for offline development and testing — it generates and reports at `raw_samplerate` too, so it exercises the same dual-rate path.

2. Preprocessing — `DataCollector` (`collector.py`)

`DataCollector.receive_data(frame)` runs in the hardware callback thread:

1. For each enabled channel, extracts the channel column from `frame['data']` (at `raw_samplerate`)
2. Converts mV → engineering units using `ScopeSensor.sensitivity` (if a sensor is assigned)
3. Optionally applies a **4th-order Butterworth highpass** filter (default 10 Hz declared band edge), at the raw rate. Causal (`sosfilt`) with filter state carried across consecutive streaming blocks, seeded from the block mean on the first block; zero-phase `sosfiltfilt` was tried and reverted — its forward+backward pass effectively doubles the order and overshoot both block edges by 35–45% for a low cutoff over a short block
4. Wraps each channel's data in a `VibeSample` at the raw rate (anti-aliasing for the *acquisition* Nyquist is no longer applied here — it happens upstream in `PicoScopeStream`, before the ADC's own Nyquist limit can fold high-frequency content into the passband). `VibeSample.data / .samplerate` are this raw signal, and are what gets written to HDF5 and what envelope analysis reads directly
5. Assembles a frame dict `{ch: VibeSample, 'overflow': mask}` and appends it to the frame ring cache (configurable depth via `AcquisitionSettings.cache_frames`, default 32)
6. Sets `new_frame_event` to signal the GUI render loop

3. Spectral Analysis — `DataCollector.process_sample()` (`collector.py`)

`DataCollector.process_sample(ch, sample)` first decimates the raw-rate `VibeSample` down to the display rate (`AcquisitionSettings.samplerate`, `maxfreq`-driven) via `decimate_to_rate()` — a rational-ratio polyphase resample (`scipy.signal.resample_poly`) reusing the same Kaiser stopband design as the hardware anti-alias filter, cached on the sample so repeated calls (browsing, unit changes) don't re-resample. Everything below runs on that decimated signal and computes and returns a `ChannelResult`:

- **Welch PSD** — `scipy.signal.welch` with a configurable window function, one segment per frame (`nperseg = blocksize`, so the delivered line count and bin width match what the UI states), and bin size controlled by `AcquisitionSettings.binsize`
- **Spectral averaging** (optional) — the *N* most recent *valid* frames up to and including the one displayed are averaged in the **power** domain. Overloaded and rate-degraded frames are rejected from the average. Since the HDF5 stores individual raw frames, averaging is recomputed on load and *N* can be changed after the fact
- **Frequency-domain integration** — when the assigned `ScopeSensor.engineering_units` modality differs from the target display unit, integration is applied by multiplying the spectrum by $(1j \cdot 2\pi f)^n$ where *n* is the number of integration steps (negative = integrate, positive = differentiate)
- **Peak detection** — `rev80.peaks.select_peaks`: a per-bin local noise floor is estimated with a running median, and array-valued `height / prominence` admit a line when it rises `peak_threshold_db` above its own neighbourhood. Ranking stays by descending amplitude; the reported value is the maximum bin's amplitude, with no interpolation or energy summation
- **Overall amplitude** — RMS/0-P/P-P over the **declared band**, not the whole block. Computed from a Hann-tapered, band-masked transform so all five integration orders describe one band
- **Crest factor and kurtosis** — computed on the band-limited displayed trace. Deliberately *not* averaged: they exist to catch the frame that is not steady

Envelope analysis reads around this. `DataCollector.eu_scaled_raw(ch, sample)` returns the highpass-filtered signal in the sensor's own engineering units *at the raw rate*, skipping the `maxfreq` decimation step entirely — this is what the Envelope tab demodulates, so a low display `maxfreq` never limits how much bandwidth a bearing-resonance search can see.

4. Visualisation — `GUI._poll_new_frames()` / `_display_frame()` (`gui.py`)

The GUI uses a manual render loop (`while dpg.is_dearpygui_running()`). Each tick, `_poll_new_frames()` checks `DataCollector.new_frame_event` . If set, it grabs the latest frame from `frame_cache` and calls `_display_frame()` :

- Calls `DataCollector.process_samples()` → list of `ChannelResult` for each enabled channel
- Updates time-domain and spectrum line series via `dpg.set_value()`
- Updates peak table and trend plot
- If multiple frames arrived since the last tick, only the newest is rendered — earlier frames remain in cache for browsing
- Axis limits are **not** automatically adjusted; press **Autoscale** to fit all axes on demand

AcquisitionSettings

`AcquisitionSettings` (`sample.py`) centralises spectrum, filter, and channel config. Use the public properties — do not write to private `_` attributes directly.

Spectrum settings

`samplerate / blocksize` are the **display** rate — what the Spectrum tab's Welch PSD is computed at, and what the Acquisition dialog's "Sample Rate" field shows. They no longer describe what the hardware actually acquires; that's the separate, fixed `raw_samplerate / raw_blocksize` pair below. `maxfreq` still only affects the display side.

Property	Description
<code>maxfreq</code>	Upper frequency of interest (Hz) — drives <code>samplerate</code> (display) selection. Clamped in the setter to what <code>raw_samplerate</code> can back ($\leq \text{raw_samplerate} / 2 / 1.28$), since nothing above that was ever captured
<code>binsize</code>	Frequency resolution of Welch FFT (Hz) — drives <code>blocksize</code> selection
<code>samplerate</code>	Derived, display rate — exactly $2.56 \times \text{maxfreq}$ (the 28% margin above $2 \times$ Nyquist gives the mandatory anti-alias filter a real transition band — same ratio commercial FFT vibration analyzers use). Every preset divides <code>raw_samplerate</code> by an exact integer, and the top preset meets it exactly, so the display rate can never exceed what was acquired. <code>DataCollector</code> decimates the raw acquisition down to this rate before computing the Spectrum tab's PSD
<code>blocksize</code>	Derived, display rate — next power of 2 satisfying $\text{samplerate} / \text{blocksize} \leq \text{binsize}$

Property	Description
<code>raw_samplerate</code>	Fixed — <code>RAW_SAMPLERATE_HZ</code> (25.6 kHz = $2.56 \times$ the 10 kHz top preset), independent of <code>maxfreq</code> . What <code>PicoScopeStream</code> actually acquires, what gets stored to HDF5, and what envelope analysis (<code>DataCollector.eu_scaled_raw</code>) reads directly. Hardware-validated on a PicoScope 4424A — see the module comment above <code>STREAMING_CEILING_HZ</code> in <code>picoscope.py</code> and <code>scripts/validate-streaming-capacity</code> for re-validating on other hardware
<code>raw_blocksize</code>	Derived, raw rate — sample count spanning the same <code>acquisition_period</code> as <code>blocksize</code> , at <code>raw_samplerate</code> . Not necessarily a power of two — it isn't a Welch segment length, just how many raw samples one frame holds
<code>acquisition_period</code>	Derived — <code>blocksize / samplerate</code> (seconds). Same value whether computed from the display or raw pair — one frame is one time window at two sample counts
<code>n_fft_bins</code>	Derived — number of spectrum lines actually displayed, DC up to <code>maxfreq</code> . Not the full one-sided transform: the band between <code>maxfreq</code> and <code>fs/2</code> is the anti-alias guard band and is not shown
<code>fft_window</code>	Welch window function: <code>'hann'</code> (default), <code>'blackmanharris'</code> , <code>'flattop'</code> , <code>'hamming'</code> , <code>'boxcar'</code> , <code>'bartlett'</code>
<code>welch_overlap</code>	Welch segment overlap fraction (default 0.5). Currently inert: <code>nperseg == blocksize</code> , so there is exactly one segment per frame
<code>band_fmin / band_fmax</code>	Declared measurement band for the overall amplitude (Hz). <code>None</code> = derive: <code>highpass_fc</code> up to <code>maxfreq</code> . ISO 20816 presets are offered in the acquisition dialog. Stored with the data — an overall taken over a different band is a different measurement
<code>averaging_enabled / n_averages</code>	Linear power averaging of the spectrum over N frames (default off, <code>N = 8</code>). Cuts noise-floor scatter as $1/\sqrt{N}$; does not lower the floor's level, and assumes the machine is steady across the window. Clamped by <code>cache_frames</code>
<code>peak_threshold_db</code>	How far a spectral line must rise above its own local noise floor to be reported (default 9.5 dB)

Filter settings

Property	Description
<code>highpass_enabled</code>	Enable the 4th-order Butterworth highpass filter. Causal (<code>sosfilt</code>) with filter state carried across consecutive streaming blocks; replayed frames are filtered statelessly from a settled initial condition, so browsing is order-independent

Property	Description
<code>highpass_fc</code>	Lower edge of the declared measurement band (Hz) — the frequency at which the response must still be within $\pm 10\%$ (ISO 2954), not the -3 dB knee. The Butterworth knee is placed below it at $f_{\text{edge}} \times (A^2/(1-A^2))^{(-1/2N)}$, which is $0.834 \times f_{\text{edge}}$ at the shipped order 4

Anti-aliasing is no longer a user-configurable lowpass. Two mandatory, automatic filters are involved: `PicoScopeStream` anti-alias filters and decimates the oversampled ADC signal down to `raw_samplerate` at capture time (fixed, not `maxfreq`-derived); `DataCollector` then anti-alias filters and decimates that raw signal again, down to `samplerate` (display, `maxfreq`-derived), before computing the Spectrum tab's PSD. There's no separate cutoff to set for either.

Channel settings

Property	Description
<code>enabled_channels</code>	List of active channel indices
<code>channel_voltage_ranges</code>	Dict <code>{ch: range_index}</code> — PS4000A voltage range per channel
<code>channel_couplings</code>	Dict <code>{ch: 'AC' 'DC'}</code> — input coupling per channel

Cache settings

Property	Description
<code>cache_frames</code>	Ring buffer depth in frames (default 32); configurable via GUI Acquisition tab

Helper methods: `voltage_range_for(ch)`, `coupling_for(ch)`, `name_for(ch)`, `target_unit_for(ch)`, `amplitude_mode_for(ch)`, `copy()`.

VibeSample and ChannelResult

VibeSample

`VibeSample` (`sample.py`) holds one block of time-domain samples for a single channel.

```

sample.data           # numpy.ndarray float64, shape (N,), raw mV samples
sample.samplerate     # int - Hz
sample.unit           # str - 'mV' (raw hardware unit)
sample.timestamp      # str - ISO format timestamp
sample.rel_time       # float - seconds since stream start
sample.status         # str - 'OKAY', 'OVERFLOW', etc.
sample.overall_ampl_by_integration_order # ndarray (5,) - broadband RMS for orders -2..+2

# Cached on first call; re-computed when Welch config changes:
sample.psd_mv         # numpy.ndarray - Welch PSD (mV RMS)
sample.freq_hz        # numpy.ndarray - frequency axis (Hz)

```

HDF5 save/load is handled by `DataCollector.save_data()` and `DataCollector.load_data()`, not by `VibeSample` directly.

ChannelResult

`ChannelResult` is a frozen dataclass returned by `DataCollector.process_sample(ch, sample)`. It is the canonical display-ready result for one channel at one instant.

```
result.channel      # int
result.unit         # str – display unit
result.time_data    # ndarray – time-domain signal in display units
result.time_vec     # ndarray – time axis (seconds)
result.freq         # ndarray – full Welch frequency axis (Hz)
result.spectrum     # ndarray – PSD in display units
result.peaks        # ndarray – indices into freq/spectrum arrays
result.overall      # float – broadband amplitude (RMS/0-P/P-P per config)
result.timestamp    # datetime
result.rel_time     # float
result.status       # str
```

The `freq` and `spectrum` arrays cover the **full Welch output range** (up to Nyquist). `maxfreq` governs samplerate selection and broadband energy integration but does not crop the displayed spectrum.

ScopeSensor

`ScopeSensor` (`scope_sensor.py`) describes an IEPE sensor connected to one PicoScope channel.

```
sensor.name         # str – user label (e.g., 'PCB 352C33 Ch1')
sensor.engineering_units # str – source modality (e.g., 'g', 'mm/s')
sensor.sensitivity   # float – mV per engineering unit (e.g., 10.2 for 10.2 mV/g)
sensor.id           # str – UUID, used as persistent key
sensor.notes        # str – freeform
```

Sensors are managed through `ScopeSensorRegistry` and assigned to channels via the GUI Channel Config panel. When a sensor is assigned, `DataCollector` divides incoming mV data by `sensitivity` to produce engineering units before creating `VibeSample` objects.

The display/integration target unit and amplitude mode (`RMS`, `0-P`, `P-P`) are **per-channel** settings (`AcquisitionSettings.channel_target_units` / `channel_amplitude_modes`), not part of the sensor definition — the same sensor can be wired to different channels with different targets.

Data Storage

Every stored frame — manual save or Monitor Mode interval/burst — is the full raw-acquisition-rate capture (`raw_samplerate` / `raw_blocksize`), not the `maxfreq`-driven display rate: storage cost is independent of whatever `maxfreq` is set to. This is what lets envelope analysis re-run at full bandwidth on an old file even if it was captured with a low `maxfreq` for the Spectrum tab. It's also why the per-year estimate in the Monitor config dialog can be substantial for a tight interval — see [Monitor Mode](#).

Manual saves (v4 format)

Single-measurement saves written by **File** → **Save** or `DataCollector.save_data()`, into `~/Documents/Rev80/data/` by default (`_paths.data_dir()`):

```
~/Documents/Rev80/data/YYYY-MM-DD-HHMMSS.h5
  /metadata/
    .attrs          version=4, notes
    acquisition/     AcquisitionSettings fields
    scope_sensors/   sensor library snapshot
    channels/{ch}/    per-channel config
  /frames/{i}/
    .attrs          timestamp, rel_time, samplerate, status
    {ch}/data        (blocksize,) float64 mV, gzip-compressed
  /trend/{ch}/
    rel_times, orders (M,5) integration orders matrix
```

```
collector.save_data(Path("~/Documents/Rev80/data/my_run.h5").expanduser())
collector.load_data(Path("~/Documents/Rev80/data/my_run.h5").expanduser())
```

Monitor sessions (v5 format)

Monitor Mode writes one `session.h5` per session, appending frames as the interval gate fires, under `~/Documents/Rev80/data/monitor/` by default (`--output` overrides this in headless mode; see [CLI Reference](#)):

```
~/Documents/Rev80/data/monitor/{session_id}/session.h5
  /metadata/
    .attrs          file_version=5, session_id, start_time, interval_s
    acquisition/     AcquisitionSettings snapshot at arm time
    scope_sensors/   sensor library at arm time
    channels/{ch}/    per-channel config at arm time
  /monitor/{N}/
    .attrs          timestamp, rel_time, samplerate, status,
                    overall_json, peaks_json
    {ch}/data        (blocksize,) float64 mV, gzip-compressed
  /burst/{burst_id}/
    .attrs          trigger_type, trigger_timestamp, trigger_rel_time,
                    burst_duration_s, max_overall_json, n_frames,
                    n_pretrigger_frames
    {frame_index}/
      .attrs        timestamp, rel_time, is_pretrigger, overall_json
      {ch}/data
  /burst.attrs      burst_list – JSON array of burst summaries
```

`session_id = "YYYY-MM-DD-HHMMSS"` (UTC). Loaded via the session browser or:

```
collector.load_monitor_session(session_h5)
collector.load_monitor_burst(session_h5, burst_id)
```

PicoScope Integration

Rev80 targets the **PicoScope 4000A series** as its primary acquisition hardware via the `picosdk` Python bindings.

Key implementation details

Component	Role
<code>FindPicoScope()</code>	Opens each unit in sequence; reads model, serial, build date, channel count via <code>ps4000aGetUnitInfo</code> ; returns dicts compatible with <code>VibeSensor(**dev)</code>
<code>PicoScopeStream</code>	Background thread: <code>ps4000aRunStreaming</code> → streaming callback → ADC → mV → accumulator → app callback

Resilience features: - USB power-source fallback: if `ps4000aOpenUnit` fails, retries with `ps4000aChangePowerSource` - Up to 5 open attempts with 1–2 s delay (extended delay after `PICO_NOT_RESPONDING`) - Watchdog: if no data arrives for 5 s, attempts up to 3 reconnect cycles before raising - Overflow logging rate-limited to once per 2 s per channel to avoid log spam

Dependencies

```
pip install picosdk
```

The PicoScope 4000A driver (`ps4000a.dll` on Windows, `libps4000a.so` on Linux) must be present on the system path.

Looking to build, package, or contribute to Rev80? See [CONTRIBUTING.md](#).